Introduction to Machine Learning in Python

Mahmoud Elsawy and Jean-Luc Bouchot

Inria

June 8-12 2026

Outline

What this course is

Overall, this course will talk about

- Data processing
- Computer vision
- Feature engineering
- Classifiers
- Python programming

This is a week long project-based course: progress at your own, distribute the work in a smart way.

What this course is not

Remember this course is not

- A proof-heavy course
- An exhaustive presentation of all methods for solving classification problems
- targeting a precise application: the image recognition problem is but just one example

Course setting

Plan:

- M./Tu./W. 8:00am-12:15pm; 13:30pm-5:30pm
- Th. 8:00am-12:15pm
- Group evaluation on Friday morning

Evaluation:

- Groups of 3 – to be chosen among yourselves and notified to us by Monday 4pm
- Work presentation : 10 minutes / group + 5 minutes of questions
- Deliverables sent to us no later than Thursday 11/06/26, 5pm, include
  - All codes as an archived file (named `LastName1_LastName2_LastName3_ML.tar` or `.zip`)
  - A brief report describing your results **No more than 10 pages**, all figures and annexes included.
  - A small presentation used as a support for the oral presentation
  - `mahmoud.elsawy@inria.fr` or `jean-luc.bouchot@inria.fr`

What is Machine Learning?

Definition

Machine learning is the art to teach an automated system to learn from previous data in order to predict on yet unseen data.

- It is a subset of artificial intelligence allowing systems to learn autonomously
- It uses a sample of the universe of potential data to learn on its own
- It enables a computer to solve complex problem which would be out of reach to traditional programming principles.

Why Are We Here? Applications of Machine Learning

Engineering & Automation

- Autonomous vehicles (How does a car know a pedestrian from a shadow?)
- Unmanned Aerial Systems (Drone flight control)
- Industrial robotics (Defect detection on assembly lines)

Prediction & Analytics

- Algorithmic trading and asset valuation
- Hyper-personalized medical diagnostics
- Scalable recommendation systems (Netflix, Amazon)

The Three Fundamental Archetypes of Learning

How Do Computational Models Actually Learn?

1. Supervised Learning (Our Focus Today)

Teaching by example. We provide the data *and* the correct answers (labels). The algorithm learns the mapping rules connecting the two.

2. Unsupervised Learning

Finding hidden structures. We provide data with *no* answers. The algorithm groups things by similarity (e.g., customer segmentation).

3. Reinforcement Learning

Learning by trial and error. An agent takes actions in an environment and learns to maximize a reward (e.g., teaching a robot to walk).

Basics of supervised learning

How does it work?

Step 1 : Some samples of data together with the labels are given (e.g. data are images, labels are *cat* or *dog*).

Step 2 : Given the right (algorithm, feature) pair, the machine detects valuable differences / characteristics for all labels.

Step 3 : Once learning has happened, the machine is able to extract relevant features from an unknown image and classify it (e.g. This picture is that of a dog).

Applications

- Email filtering (Spam vs Non-Spam).
- House price prediction.
- Voice recognition (e.g. : Siri, Alexa).

Supervised learning: The various types

Depending on the types of labels, we distinguish

- Classification (a.k.a. logistic regression) happens when the labels are categories (non numerical). What are examples of such problems?
- We talk about regression when the labels are continuous (numerical) data. Name a few examples.
- When the labels are discrete ordered sets, we talk about ordinal classification or regression. What are examples of such problems?

Assume you try to predict a hand written number for reading zip codes. What type of supervised learning would you consider?

Basics of unsupervised learning

How does it work?

Step 1 : Gather many unlabeled data.
Step 2 : The system groups *similar* data together (it is called clustering).
Step 3 : The clusters are analysed and interpreted by a human.

Applications

- Customers segmentation (groups of users or customers).
- Fraud detection in financial systems.
- Automated photos sorting.

Basics of reinforcement learning

How does it work?

Step 1 : The machine explores various actions / paths.
Step 2 : It receives a reward or a penalty depending on the outcome of its exploration.
Step 3 : It adapts its strategy as the number of trials increases.

Applications

- Videogames (e.g. AlphaGo).
- Autonomous robots / vehicles (drones, self driving cars).
- Algorithmic trading.

What is this week about?

Fine grained classification

Pred: Kerry_blue_terrier (0.24)
True: Kerry_blue_terrier

Pred: chow
True: chow

Going through an AI project step by step

An IA project is articulated around a few phases

- Pre-analysis: Before launching a project, one investigates the expected costs, human and hardware needs, ... – Skipped today
- Data collection and labeling – Skipped today.
- Data exploration: important step during which missing data and outliers are handled. – Skipping the outlier / missing data handling this week.
- Features engineering
- Learning model design
- Actual learning and validation

It is often mandatory to go back a few steps and adapt some previous parts.
This week's goal is to lead an AI project from end to end, iterating over the feature engineering and modeling parts to reach the best accuracy possible.

Our Engineering Challenge: Fine-Grained Classification

The Dataset: SmallDB

- Derived from the famous **Stanford Dogs Dataset**.
- **The Goal:** Teach the computer to distinguish between canine breeds (e.g., Chihuahua, Pug, Malamute, Beagle).
- **Why is this hard?**
  - *Low inter-class variance:* Dogs share the same basic biology (eyes, nose, ears).
  - *High intra-class variance:* Photos have crazy lighting, different poses, and messy backgrounds.

How the Computer Sees Images

An image may have various representations:

- A grayscale image isn't a picture; it's a matrix of intensity values from 0 (black) to 255 (white): $X \in \{0, \cdots, 255\}^{H \times W}$
- A colour image usually has three channels; it is now represented as a rank 3 tensor

$$X \in \{0, \cdots, 255\}^{H \times W \times 3}$$

- The channels aren't always colours!
- The intensities might be 8-bit integers or some floating point after normalization.
- What representation is used in the given code?

Data preparation: image cropping

Extracting a dog

If you browse the images you will notice:

- Some very diverse background
- Potentially more than one dog per image
- Potentially various dog breeds on the same image
- Various sizes

The function `read_and_crop` provided extract **one** dog from every image automatically! Make sure you understand this function for future needs

Original Asset Image Frame

XML Isolated Target Region

Step 1: Data Exploration – The "Lazy Algorithm" Trap

Remember Supervised Learning?

- Our goal is to learn a mapping: $f(\text{Pixels}) \approx \text{Dog Breed}$.
- But algorithms are mathematically "lazy." They take the path of least resistance to minimize their error score.

The Accuracy Illusion

Imagine a dataset with **90 pictures of Pugs** and **10 pictures of Beagles**.

To get 90% accuracy, the model doesn't need to look at the pixels at all! It simply guesses "Pug" every single time.

The Core Rule: Before we train any model, we must prove our dataset is fair and balanced. Otherwise, the model learns the distribution, not the images.

How Do We Measure "Fairness"?

We need an automated metric.

We can't always look at bar charts manually. We need a mathematical score to tell our Python pipeline if the dataset is healthy. For this, we borrow a concept from physics and information theory: **Entropy**.

Entropy = Unpredictability (Surprise)

- **A rigged coin** (heads on both sides) has no surprise. You know the outcome. **Entropy is 0.**
- **A fair coin** (50% heads, 50% tails) has maximum unpredictability. **Entropy is High.**

Applying it to SmallDB

- If our dataset is mostly Pugs, the "surprise" of pulling a Pug is low. (Low Entropy = Imbalanced / Bad).
- If all 4 dog breeds have exactly 25% of the photos, surprise is maximized! (High Entropy = Balanced / Good).

The Math: Shannon Information Entropy

The Formula

$$H(P) = -\sum_{i=1}^{K} p_i \log_e(p_i)$$

- K = Total number of classes (e.g., 4 breeds).
- p_i = The empirical probability (percentage) of breed i in the dataset.

Theoretical Bounds

$$0 \leq H(P) \leq \log_e(K)$$

- If $H(P)$ approaches 0, your pipeline should throw a critical warning!
- If $H(P)$ is close to $\log_e(K)$, your data is perfectly balanced.

Lab Task 1: Building our Balance Tracker

TASK 1: Implement Shannon Information Entropy

Open the student workbook and locate the `entropy(p)` function. Let's translate the math into robust Python code to check our SmallDB health.

Engineering Constraints to Watch Out For:

1. **Normalization:** Ensure your raw image counts (p) are converted into probabilities (they must sum to 1.0).
2. **The** $\log(0)$ **Crash:** If a class accidentally has zero samples, $\log_e(0)$ evaluates to negative infinity and will crash your script. Use a boolean mask (e.g., `p = p[p > 0]`) to filter out empty classes safely.

Once completed, your code will generate `figures/class_distribution.png`!

Step 2: Preprocessing – The Uniform Matrix Constraint

The Real World is Messy

- If you download pictures of dogs from the internet, they come in infinite shapes and sizes (e.g., 1080×1080, 800×600, 400×900).

Math is Rigid

- Machine learning models (like our k-NN classifier) require fixed-length vectors.
- We cannot feed a 1-million pixel image and a 400-thousand pixel image into the same equation.
- **Requirement:** Every single image must be forced into a uniform grid (e.g., 64×64).

The Naive Solution

Just use a basic image resize function: `resize(image, (64, 64))`.

Question for the class: What happens to a rectangular image when you force it into a perfect square?

The Distortion Trap: Destroying Biological Geometry

The Problem with "Squishing"

- Naive canvas resizing stretches biological proportions to force the image to fit.
- **The Engineering Failure:** Distance-based classifiers look at shapes. If you vertically squash a tall, skinny Greyhound, it becomes mathematically identical to a long, low Dachshund!
- By naively resizing, we are actively destroying the biological markers we are trying to teach the model to find.

Notice how the "Stretched" version completely changes the shape of the dog's head.

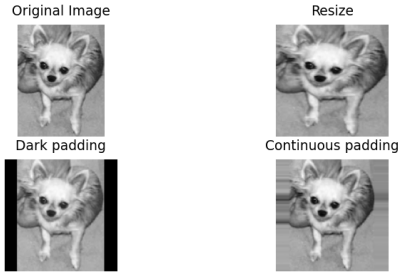
The Solution: Aspect-Preserving "Letterboxing"

How do we resize without stretching?

We borrow a trick from the film industry. When you watch an old rectangular movie on a modern widescreen TV, the TV doesn't stretch the actors. It scales the movie until it hits the top and bottom of the screen, and fills the sides with black bars (*padding*).

The Algorithmic Logic

1. Calculate how much we need to shrink/zoom the height and width.
2. Find the **limiting axis** (the side that needs to shrink the most to fit inside our 64×64 box).
3. Shrink *both* sides by that exact same ratio.
4. Paste the new image in the center of a matrix.
5. Potential padding: black, white, extension of the last pixel, mirroring...



Lab Task 2: Aspect-Preserving Scaling

TASK 2: Translate the "Letterboxing" Logic into Code

Navigate to the `resize_and_pad` function block in your workbook. You must write an algorithm that scales the image without stretching it.

Your Implementation Steps:

1. Compute the height scaling ratio (`h_r`) and width scaling ratio (`w_r`).
2. Use an `if` statement to determine which ratio is smaller (the limiting axis).
3. Calculate the new, proportionally safe dimensions (`new_h, new_w`).
4. Calculate the exact pixel offsets (`top, bottom, left, right`) to center the image.

Canvas Slicing Hint

We embed your proportionally scaled image onto a blank background canvas using matrix slicing:

```
output_img[bottom:top, left:right] = resized_img
```

Step 3: Feature Engineering – The Raw Pixel Trap

Why Aren't We Done Yet?

- In Step 2, we perfectly centered our dogs into 64×64 matrices. That gives us 4,096 pixel values.
- **The Question:** Why can't we just feed those 4,096 numbers directly into our classifier?

Pixels Measure Light, Not Shape

- Imagine a black Pug in a dark room vs. a white Pug in the bright snow.
- To a computer looking at raw pixels, these matrices are 100% mathematically different. It thinks they are entirely different objects.

The Cartoon Solution

How do humans recognize objects? We don't need color or lighting. We can recognize a dog from a simple line drawing.

We look for **edges and outlines**. We need to teach the computer to do the same.

PCA Theory: Foundation and Motivation

What is PCA?

- Principal Component Analysis (PCA) is a **linear dimensionality reduction** technique.
- It finds the directions of maximum variance in your data.
- These directions become your principal components: ordered by importance.

The Core Idea

Start with a data matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ containing n samples and d features (e.g., pixel values).
We want to find a smaller set of features that capture the most *information* (variance).
Goal: Project onto k dimensions where $k \ll d$, losing as little information as possible.

Mean Centering: The First Step

Before PCA, we extract the mean: $\mathbf{X}_{\text{centered}} = \mathbf{X} - \boldsymbol{\mu}$ where $\boldsymbol{\mu}$ is the sample mean. This centers the data at the origin, ensuring the principal components pass through the center of the cloud.

PCA Computation: Eigenvalue Decomposition of the Covariance Matrix

The Mathematical Recipe

1. **Compute the Covariance Matrix:**

$$\mathbf{C} = \frac{1}{n-1}\mathbf{X}_{\text{centered}}^T\mathbf{X}_{\text{centered}} \in \mathbb{R}^{d\times d}$$

It captures how features vary together.

2. **Eigenvalue Decomposition:**

$$\mathbf{C} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$$

where $\mathbf{V}$ are the principal directions and $\mathbf{\Lambda}$ are the variances along each direction.

3. **Sort by Variance:** Rank eigenvalues in descending order. Larger eigenvalues = more important components.

Projection onto Principal Components

Once we have the top k eigenvectors (columns of $\mathbf{V}_k$), we project:

$$\mathbf{Z} = \mathbf{X}_{\text{centered}}\mathbf{V}_k$$

This gives us k new features with maximum variance!

Computational Cost:

- Computing $\mathbf{C}$: $\mathcal{O}(nd^2)$
- Eigendecomposition: $\mathcal{O}(d^3)$
- Problem: If d is large (millions of pixels), this is expensive!

PCA via SVD: The Efficient Alternative

Why SVD is Better

- Instead of computing $\mathbf{C}$ explicitly, we decompose $\mathbf{X}_{\text{centered}}$ directly:

$$\mathbf{X}_{\text{centered}} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

- $\mathbf{U}$: left singular vectors (sample space)
- $\mathbf{\Sigma}$: singular values (in descending order)
- $\mathbf{V}$: right singular vectors = PCA eigenvectors!

The Connection

The covariance matrix can be expressed as:

$$\mathbf{C} = \frac{1}{n-1}\mathbf{V}\mathbf{\Sigma}^2\mathbf{V}^T$$

So the eigenvalues of $\mathbf{C}$ are $\lambda_i = \frac{\sigma_i^2}{n-1}$.

The PCA projection becomes:

$$\mathbf{Z} = \mathbf{U}\mathbf{\Sigma}$$

Computational Advantage:

- SVD computational cost: $\mathcal{O}(ndk)$ for extracting top k components (can be much faster when $k \ll d$).
- Modern SVD implementations use incremental/randomized methods, making PCA tractable even for high-dimensional data.

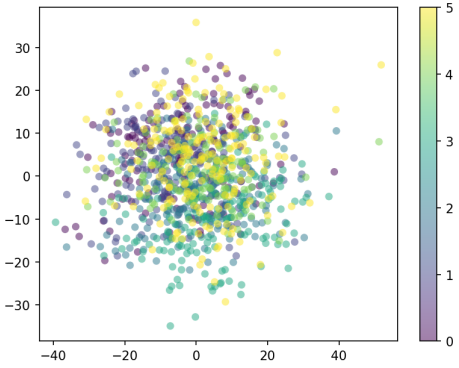
More on data exploration: Visualizing the database globally

Problem: From our 64×64 images, we cannot illustrate the database globally.
Solution: Apply dimension reduction techniques to lower the dimensionality to 2 or 3.

Fill out all TASK 3 from the `Python` script to generate such a figure.

Dimensionality reduction with Principal Component Analysis (PCA)

- Concentrate the global pixel information.
- Transform a $H \times W$ dimensional image into few dimensions (2 or 3 for visualisation purposes).
- Preserve most of information along the main *principal components* (aligned with the highest variance).



Step 3: Feature Engineering – The Raw Pixel Trap

Why Aren't We Done Yet?

- In Step 2, we perfectly centered our dogs into 64×64 matrices. That gives us 4,096 pixel values.
- **The Question:** Why can't we just feed those 4,096 numbers directly into our classifier?

Pixels Measure Light, Not Shape

- Imagine a black Pug in a dark room vs. a white Pug in the bright snow.
- To a computer looking at raw pixels, these matrices are 100% mathematically different. It thinks they are entirely different objects.

The Cartoon Solution

How do humans recognize objects? We don't need color or lighting. We can recognize a dog from a simple line drawing.

We look for **edges and outlines**. We need to teach the computer to do the same.

Lab 4: Feature engineering 1. PCA features

PCA is a powerful tool for data approximation.
Look at TASK 4 and fill out the code to show the approximation capability of PCA.
It can be used for data compression, approximation, representation, visualisation, ...
You should be able to generate a figure similar to the one on the right.

Feature engineering 1. PCA features

Single PCA: database visualisation

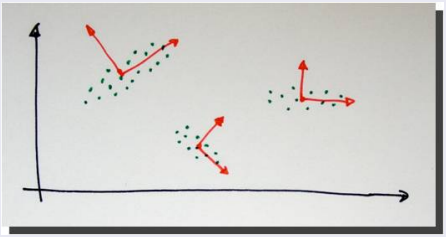
Using the PCA means

- One model fits all: all classes are mixed into the soup.
- Detects patterns relevant to all classes
- Not necessarily good for classification / identification
- Good for global visualisation

Multiple PCAs: feature computations

Using one PCA per class means:

- Specific models for specific class
- Concentrated PCs for every class
- (Hope:) High correlation of images with PCs of true labels, low with other classes



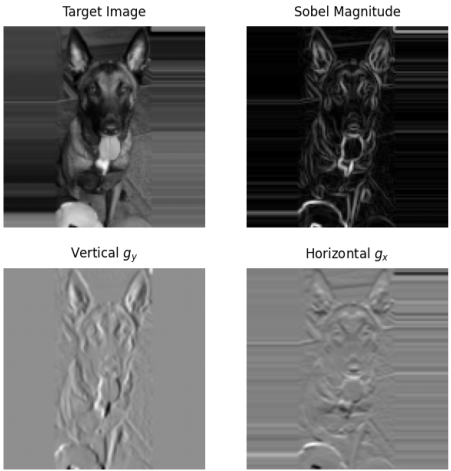
From Light to Shape: The Concept of Gradients

How does Math find an "Edge"?

- An edge is just a place where the image brightness changes suddenly (e.g., dark fur against a light wall).
- In calculus, the rate of change is called a **Gradient**.
- By calculating gradients, we strip away the lighting and keep only the structural outlines.

The Sobel Filter

We use mathematical filters (called Sobel filters) to scan the image horizontally (g_x) and vertically (g_y) to find these sudden jumps in pixel intensity.



Notice how the Sobel magnitude reveals the outline of the dog, completely ignoring the flat

Histogram of Oriented Gradients (HOG) Explained

What does HOG actually do?

We don't just want a messy web of random edges. We want a summary of the shape. HOG acts like a "Directional Vote Counter" for different parts of the image.

The HOG Step	What it means in plain English
1. Spatial Grid Partitioning	Chop the 64×64 image into a grid of smaller boxes (so we know *where* the shapes are, e.g., ears at the top).
2. Calculate Gradients	Find the edges in the box.
3. Angular Mapping	Are the edges horizontal, vertical, or diagonal? (We ignore light-to-dark vs dark-to-light polarity).
4. Binning & Accumulation	Tally up the "votes." (e.g., Box 1 has 50 strong vertical edges and 2 weak horizontal edges).

Lab Task 5: Building the Directional Vote Counter

TASK 3: Complete the Spatial Grid HOG Accumulation Loops

Navigate to the `compute_hog` function. You must complete the nested loops that act as the "vote counters" for our edge directions.

Implementation Logic:

1. Loop over the rows and columns of your sub-cells.
2. For every single pixel inside that specific cell, check its gradient angle.
3. Convert that angle into a specific "Bin" index (e.g., 0° to 20° goes into Bin 0).
4. Add the pixel's edge strength (**Magnitude**) into that bin. Stronger edges get louder votes!

Code Safety Hint

Watch out for edge cases where the math perfectly hits the upper boundary:
`angle // bin_width == nb_bins`
Force this index to snap back to `nb_bins - 1` to prevent out-of-bounds array crashes.

Step 4: The Assembly Line – Creating a "Fingerprint"

Putting the Pieces Together

- **PCA** gave us the global, structural shape of the dog.
- **HOG** gave us the local, directional edge contours.
- Neither is perfect on its own. We need to combine them to give the computer the full picture.

Feature Concatenation

- We literally glue the two mathematical arrays together end-to-end.
- This combined vector is the unique **"Mathematical Fingerprint"** for that specific dog.

The Fingerprint Matrix

$$X_{\text{combined}} = [\Phi_{\text{PCA}} \,||\, \Phi_{\text{HOG}}]$$

If PCA has 5 values per class, 6 classes, and HOG has 128 values, our new fingerprint has 158 values representing the dog.

The "Bully" Problem: Why We Must Scale Features

Distance Math is Sensitive

- Imagine predicting someone's health using their Weight (e.g., 75,000 grams) and Height (e.g., 1.7 meters).
- If we calculate the distance between two people, the 75,000 will mathematically erase the 1.7. The height gets completely ignored!

The Solution: Standardization We force every feature to speak at the same "volume" (Mean of 0, Variance of 1):

$$z = \frac{x - \mu}{\sigma}$$

The Machine Learning Equivalent

PCA outputs tiny decimals (e.g., 0.15). HOG outputs large sums (e.g., 450.0). If we don't fix this, HOG will mathematically "bully" PCA out of the equation.

Now, a 1.0 in PCA carries the exact same mathematical weight as a 1.0 in HOG.

The Classifier: k-Nearest Neighbors (k-NN)

How does the model actually guess the breed?

k-NN is the "Show me your friends and I'll tell you who you are" algorithm. It has no brain; it relies purely on spatial distance.

The Classification Steps:

1. Take the new, unknown dog's fingerprint.
2. Plot it in our 133-dimensional math space.
3. Measure the distance to every other dog in our training database.
4. Find the $k = 5$ closest dogs.
5. Take a majority vote (e.g., 3 Pugs, 2 Beagles $\rightarrow$ The model guesses "Pug").

Why Step 2 (Scaling) was so critical:

If we didn't scale the data, the spatial distances would be entirely warped by the largest numbers, and the "closest neighbors" would be totally wrong!

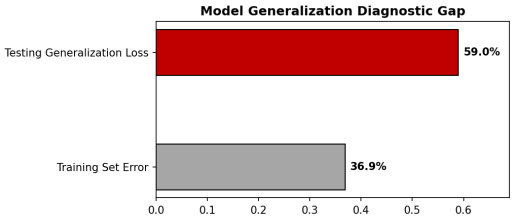
Did the Model Learn, or Did It Cheat? (Overfitting)

The "Practice Test" Analogy

- **Training Data** is the Practice Test. The model sees it over and over.
- **Testing Data** is the Final Exam. The model has *never* seen these images before.

The Generalization Gap

- If you score 100% on the practice test, but 50% on the final exam, you didn't learn the concepts. You just memorized the practice answers.
- In Machine Learning, this failure to generalize is called **Overfitting**.



We evaluate this using Hamming Loss. We want both error bars to be low and close together.

Lab Task 4: Proving the "Bully" Effect

TASK 4: Diagnostics & Feature Scaling Critical Reflection

You will now prove that distance-based classifiers fail if data is not scaled.

Your Implementation Steps:

1. Run your completed pipeline normally (with the `StandardScaler` active).
2. Note your Training Error vs. Testing Error in your report.
3. Now, **comment out** the lines applying `StandardScaler`.
4. Pass the raw, unscaled X_{combined} directly into the k-NN classifier.
5. Run the script again. Record the catastrophic change in your Testing Error!

Question for your Report:

Why did the Generalization Gap explode when the scaler was turned off? Which feature (PCA or HOG) dominated the k-NN distance calculation?

Lab Assignment Summary & Deliverables

Phase 1: Code Implementation (The Python Workbook)

Ensure your script runs without errors and generates all `figures/`.

- **Task 1 (Math):** Implement the Shannon Entropy formula.
- **Task 2 (Geometry):** Code the aspect-preserving "letterbox" padding.
- **Task 3 (Vision):** Build the HOG spatial grid "vote counter" loops.
- **Task 4 (Diagnostics):** Run the final pipeline, then *disable* the scaler and run it again.

Phase 2: The Engineering Report

- **Analysis 1 (Data):** Based on your Entropy score, explain how severe class imbalance manipulates a distance-based classifier's decisions.
- **Analysis 2 (Geometry):** Explain the exact mechanical failure that occurs in distance math if we stretch images rather than padding them.
- **Analysis 3 (Vision):** Why do we strictly constrain gradient angles to $[0, \pi)$ rather than $[0, 2\pi)$? (Hint: Think about shadows and light polarity).

Core Vocabulary: Setting the Ground Rules

What is the difference between Classification & Regression?

- **Classification:** Predicting a discrete bucket, label, or group name.
  - *Other Names:* Categorization, Pattern Recognition.
- **Regression:** Predicting a continuous numerical trend line (e.g., estimating a dog's precise weight).

What do we call Clustering?

- Clustering is **Unsupervised Learning**.
- Unlike classification where we provide breed names, clustering gives the computer raw photos with **no target answers**.
- The computer uncovers hidden structures by sorting photos into natural visual groups on its own.

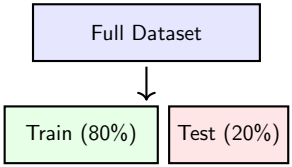
Theory 1: Data Splits & The Role of Stratification

Why is it important to split the data into Train and Test?

- If a model checks its accuracy on its own training photos, it acts like a student who memorized the practice test answers.
- The test split evaluates true out-of-sample **generalization**.

What is the role of Stratification?

- When dog breeds are **imbalanced**, random splits can leave a minority class out of the test set entirely.
- Stratification mathematically preserves population class ratios across all data slices.



Stratification Check:
Class ratios must match exactly

Lab Task 1: Quantifying the Split

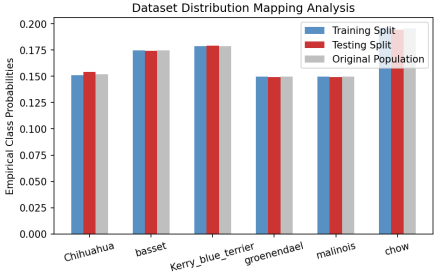
TASK 1: Did our split work?

Open your Python workbook at Section 1. We used `stratify=labels` inside `train_test_split`. Let's write the validation tool to check if the split is mathematically valid.

Your Code Assignment:

1. Calculate the empirical class probabilities for `y_train` and `y_test` using the counts from `np.unique`.
2. Complete the math for the **Cosine Similarity** score to quantify split alignment:

$$\text{Similarity} = \frac{\mathbf{p}_{\text{train}} \cdot \mathbf{p}_{\text{test}}}{\|\mathbf{p}_{\text{train}}\| \|\mathbf{p}_{\text{test}}\|}$$



Target Output: *If your similarity score is close to 1.0, the blue (Train) and red (Test) bars will match in height.*

Theory 2: Data Leakage & The Pipeline Assembly Line

What is Data Leakage?

- If you compute data-wide scaling boundaries or run global PCA before splitting your dataset, your train loop secretly incorporates structural clues from your test images.
- This turns your test split into an invalid "time machine" that hides performance failures.

The Solution: Scikit-Learn Pipelines

- `Pipeline`: Encapsulates data steps into a single conveyor belt.
- `fit()`: Extracts rules (e.g., PCA dimensions) using *only* the training partition.
- `transform()`: Applies those saved rules to test pictures blindly.

`FeatureUnion` Parallelization

Combines distinct feature generation passes (e.g., class-specific PCA structures and hand-computed HOG edges) in parallel and horizontally stacks their vectors.

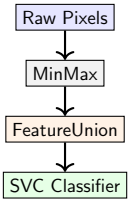
Lab Tasks 2 & 3: Building the Pipeline

TASKS 2 & 3: Build a Leak-Proof Factory

Navigate to Section 2 of your workbook. You will wrap your feature processing blocks inside standard Scikit-Learn custom estimators.

Your Code Assignment:

1. **Task 2:** Code the class loops inside `PCAInfoPreprocessing.fit()` to store distinct subspace configurations without exposing test blocks.
2. **Task 3:** Nest your custom classes within a compound `FeatureUnion` step named `all_features`.
3. Assemble your final structural workflow using the global variable name `clf`:



`MinMaxScaler` → **all_features** → `StandardScaler` → `SVC`

Theory 3: (Linear) Large margin classifiers

What is the mathematical foundation of SVMs?

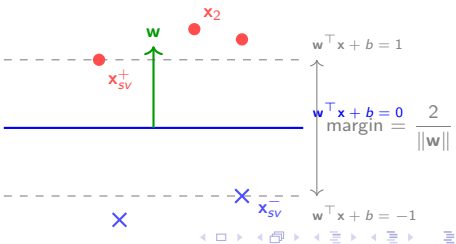
Goal: find the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that **maximizes the margin** while forbidding classification errors. **Hard Margin SVM (Linearly Separable Case):**

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2$$

$$\text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i = 1, \ldots, n.$$

Geometric Intuition

The SVM seeks the widest street (margin) separating two classes.



Theory 4: The (soft) SVM Optimization Model

What is the mathematical foundation of SVMs?

Support Vector Machines solve a constrained optimization problem to find the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that **maximizes the margin** while minimizing classification errors.

C-SVM (Soft Margin)

Solves the primal problem:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

- $C > 0$ controls **slack penalty**
- Small C: tolerate more errors (wider margin)
- Large C: fewer errors (tighter margin)

ν-SVM (Proportion Control)

Alternative formulation:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + \frac{1}{n\nu}\sum_{i=1}^{n}\xi_i \quad \text{s.t.} y_i(\mathbf{w}^\top \mathbf{x}_i +$$

- $\nu \in (0, 1]$ is the **support vector proportion**
- ν bounds the fraction of margin vi
- More interpretable than C

Theory 5: Kernel SVM and the Kernel Trick

What if data is not linearly separable?

Use a **kernel** to implicitly transform your data into a high-dimensional feature space where separation becomes linearly possible.

Common Kernels:

- **Linear:** $K(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\top \mathbf{y}$
- **Polynomial:** $K(\mathbf{x}, \mathbf{y}) = (\gamma \mathbf{x}^\top \mathbf{y} + r)^d$
- **RBF:** $K(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|^2)$

Why is the RBF kernel a good candidate

For RBF kernel with parameter γ:

$$K(\mathbf{x}, \mathbf{y}) = e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2}$$

- Small γ: **smooth**, global influence
- Large γ: **local**, each point affects nearby samples
- Implicitly infinite-dimensional transformation

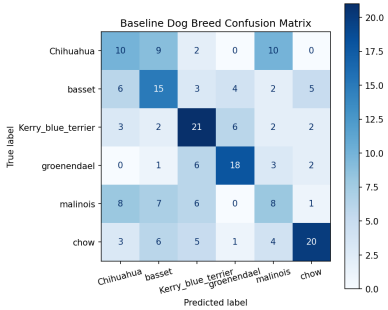
Theory 6: Support Vector Classifiers (SVC)

What is our alternative pattern classification engine?

Instead of counting local neighbor distances (k-NN), a Support Vector Machine finds an **optimal hyperplane decision boundary** that cuts through space to maximize the separation gap (margin) between opposing classes.

Linear separation versus RBF spaces

- When features blend together directly, a linear boundary cannot divide the samples.
- **Kernels** transform features into infinite dimensions to find clean geometric slices.
- We change our parameter setting using `clf.set_params(classifier__kernel='rbf')` to map complex structural patterns.



The baseline confusion matrix uncovers which

Theory 5: Tuning vs. Learning & Cross-Validation

Hyperparameter Tuning versus Learning

- **Learning (Parameters):** Spatial coefficients, bias margins, and weights calculated automatically during `.fit()`.
- **Tuning (Hyperparameters):** Architectural dials set by the designer *before* learning begins (`n_components`, Cost parameter `C`, RBF width `gamma`).

What do we mean by Cross-Validation?

- We isolate our true validation data by splitting the training group into K uniform partitions (`KFold`).
- The loop trains models on $K-1$ slices and tests on the remaining slice, cycling through all groups.

The Risk of Omitting Cross-Validation

Without K-Fold checks, tuning dials directly against a static test set leads to **Validation Overfitting**. You end up picking parameters that are lucky for that specific split but fail in production.

Lab Task 4: Automated Tuning Fields

TASK 4: Configuring validation grids inside `GridSearchCV`

Open Section 4 of your workbook. We will instantiate a grid mapping directly to our internal pipeline step names.

Your Code Assignment:

Formulate a Python dictionary named `param_grid` that matches your step parameters exactly:

- `features__pca__n_components`: $[5, 10, 20]$
- `classifier__C`: $[0.1, 1, 10]$
- `classifier__gamma`: $[0.01, 0.05, 0.1, 0.25, 0.5]$

Discussion Corner:

What happens if we increase our fold partitions (K) from 5 to 10? What is the mathematical trade-off between execution overhead and verification stability?

Theory 7: Multiclass Strategies

Teaching a Binary Classifier to Read 4 Categories

Support Vector Machines are fundamentally binary models. They can only evaluate single separations (Yes/No splits). Is tournament mapping applicable to any classifier? No! Many algorithms process multiclass targets natively. For our SVC, we need an abstraction wrapper.

One-vs-One (OvO) Strategy

- Builds pairwise match models for every combination: $\frac{C(C-1)}{2}$ total systems.
- Each sub-classifier trains on clean, small data subsets.
- Robust against class imbalances.

One-vs-Rest (OvR) Strategy

- Trains C separate models where each category fights the remaining database grouped as a single class.
- Highly efficient for broad classification problems.
- Prone to optimization errors if the groups are heavily imbalanced.

Lab Task 5: Strategic Structural Evaluation

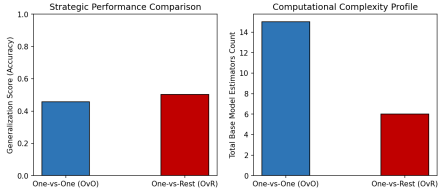
TASK 5: One-vs-One against One-vs-Rest Reductions

Open Section 5 of your workbook. We will construct two isolated system architectures to test performance profiles.

Your Code Assignment:

1. Wrap your pipeline steps inside a `OneVsOneClassifier` and record your diagnostic counts.
2. Duplicate this process using a `OneVsRestClassifier` target block.

Analysis Prompts: How do execution footprints vary across these strategies? Which architecture becomes unmanageable if our dataset grows to 1,000 distinct classes ($C \to 1000$)?



Analyze the relationship between performance accuracy and trained model counts.

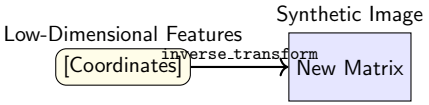
Bonus Engineering Insight: PCA as a Data Generator

Can an dimensionality reduction tool invent new data?

Yes! Because our projection rules are linear matrix transformations, we can use PCA in reverse to generate synthetic training assets.

The Generative Inverse Pipeline

- Instead of projecting pixels down to lower features, we can select artificial values inside our low-dimensional feature space.
- Running `pca.inverse_transform()` maps those numbers back to the original space.
- This reconstructs a brand-new, synthetic 64×64 dog canvas based on the learned principal patterns.



Module 2: Final Lab Deliverables

Required Report Analyses

Task 1 (Splitting): Analyze how a low Train/Test Cosine Similarity affects accuracy validation. What happens if the training split fails to capture the test domain?

Task 3 (Pipelines): Defend the implementation of automated pipelines. How do explicit `fit` and `transform` separations isolate validation metrics?

Task 4 (Validation): Contrast hyperparameter tuning with standard model training. Why must tuning metrics depend on internal cross-validation blocks?

Task 5 (Multiclass): Mathematically prove why One-vs-One complexity scales quadratically compared to One-vs-Rest architectures as $C \to \infty$.

Advanced Exploration: Transfer Learning

Pretrained Feature Extraction

- Extract **SIFT descriptors** from dog images using `opencv` or `scikit-image` (or use the ones provided), then train SVM on SIFT feature vectors (128-dim or higher).
- Download pretrained **VGG16** (ImageNet weights), remove the classification head, and use intermediate layer activations as features (e.g., last pooling layer outputs).
- Compare performance across methods: hand-crafted PCA vs. SIFT vs. VGG embeddings. Which generalizes best to unseen images? How easy is it to adapt to new breeds?

Feature Engineering Extensions

- Experiment with **HOG** features' parameters combined with PCA.
- Test **color representation**: use HSV data representation instead of RGB.

Advanced Exploration: Parameter Tuning & Sensitivity

Hyperparameter Analysis

- Create **learning curves**: plot train/validation accuracy vs. C on a logarithmic scale to identify underfitting/overfitting regions.
- Perform **kernel comparison**: evaluate linear, polynomial (degrees 2–5), and RBF with varying γ. Visualize decision boundaries in 2D PCA projections.
- Study **support vector density**: how does the fraction of support vectors (out of n) scale with C? Link margin width to generalization.

Robustness & Scaling Strategies

- Test **class weights** (`class_weight='balanced'`) to handle potential breed imbalance in the dataset.
- Compare preprocessing: `StandardScaler` vs. `RobustScaler` vs. `QuantileTransformer`. Measure SVM accuracy impact.
- Investigate **soft vs. hard margin** behavior via a sweep of C across 10 orders of magnitude (10^{-3} to 10^{6}).

Advanced Exploration: Interpretability & Benchmarking

Model Interpretability

- Visualize the **weight vector w** in the original or PCA feature space to identify which features most influence separation.
- Analyze **support vectors**: which image characteristics dominate the decision boundary? (Ambiguous samples? Breed-edge cases?)
- Implement **LIME** or **SHAP** explainability to understand individual SVM predictions on test samples.

Comparative Benchmarking

- Train alternative classifiers **(k-NN, Logistic Regression, Random Forest, Gradient Boosting)** on identical features; compare ROC curves and F1 scores.
- Build an **ensemble** via voting or stacking; does SVM boost overall accuracy when combined with other models?
- Measure **inference time** and **memory footprint** for deployment scenarios.